

Memristor-Inspired Neuromorphic Control for Robotics:

A Comparative Study of SNN and DNN Architectures with FPGA-Accelerated Synaptic Emulation

Undergraduate Thesis Proposal — Electrical & Computer Engineering

Abstract

This thesis proposes a hardware-software co-design framework for neuromorphic robotic control, combining spiking neural networks (SNNs) with memristor-inspired synaptic plasticity and FPGA-accelerated weight update logic. A leaky integrate-and-fire (LIF) SNN with *reward-modulated* spike-timing-dependent plasticity (R-STDP) is deployed in a simulated obstacle-avoidance task (PyBullet) and evaluated against a trained deep neural network (DNN) baseline and a reinforcement-learning (RL) policy; a purely Hebbian STDP variant is retained as an ablation. The FPGA implements the parallel synaptic conductance update array in hardware, a topology and update rule that is structurally analogous to the crossbar-array dynamics of physical memristive devices (a memristor-*inspired* accelerator, not a device-level emulator). The central hypothesis is that online, reward-gated synaptic adaptation—impossible in a frozen DNN—enables measurably faster performance recovery when the environment changes mid-episode, while the FPGA acceleration provides latency advantages on the weight-update kernel that justify the hardware-software co-design perspective. Metrics include response latency, synaptic operation count (SynOps), adaptation recovery time, and robustness under sensor noise.

1. Motivation and Research Gap

Modern robotic controllers are overwhelmingly based on deep neural networks trained offline: once deployed, their weights are frozen. This works well in static environments but introduces a fundamental brittleness—any distributional shift (new obstacles, sensor degradation, lighting changes) requires costly retraining. Biological neural systems solve this through *online synaptic plasticity*: the synapse itself carries memory and continues adapting throughout the lifetime of the organism.

Memristive devices are the closest physical analog to biological synapses. Their resistance state encodes a weight, updates depend on the history of current flow (state retention), the relationship is nonlinear and bounded (saturation), and device fabrication introduces stochasticity—all hallmarks of biological synaptic behavior [1]. An FPGA crossbar-style update array is the most accessible way to emulate the *structure* of this parallel, in-memory computation without fabricating physical memristor hardware; it reproduces the topology and update locality of a crossbar, not the continuous-time device physics.

Core Research Questions

RQ1. Does an SNN with online, reward-modulated STDP plasticity recover navigation performance significantly faster than a frozen DNN when the environment changes mid-episode?

RQ2. Does FPGA-parallel synaptic update logic reduce weight-update latency compared to sequential CPU/GPU computation, and at what network sizes does this advantage actually appear?

RQ3. What is the energy-efficiency tradeoff (SynOps vs. multiply-accumulate operations) between the SNN and DNN approaches at equal task performance?

2. Background

2.1. Memristive Devices and Synaptic Emulation

A memristor is a two-terminal passive element whose resistance $R(t)$ is a function of the history of current through it [1]. Its conductance $g = 1/R$ evolves as:

$$\frac{dg}{dt} = f(g, V) = \alpha V(t) [g_{\max} - g] \cdot \mathbf{1}_{V>0} - \beta V(t) [g - g_{\min}] \cdot \mathbf{1}_{V<0} \quad (1)$$

where α, β are potentiation/depression rates, and $[g_{\min}, g_{\max}]$ bounds the conductance. This naturally implements bounded, nonlinear, state-dependent weight updates—the same qualitative properties (boundedness, history-dependence, locality) that we model in software and reproduce *structurally* on FPGA. The discrete-time digital implementation does not reproduce the continuous-time differential dynamics of Eq. (1), and this distinction is maintained throughout the thesis.

2.2. Leaky Integrate-and-Fire Neuron

Each neuron in the SNN follows the LIF model:

$$\tau_m \frac{dV_{\text{mem}}(t)}{dt} = -V_{\text{mem}}(t) + R_m I_{\text{syn}}(t) \quad (2)$$

where τ_m is the membrane time constant, R_m is membrane resistance, and I_{syn} is the total synaptic input current. A spike is emitted and the membrane is reset when $V_{\text{mem}}(t) \geq V_{\text{th}}$:

$$s(t) = \begin{cases} 1 & \text{if } V_{\text{mem}}(t) \geq V_{\text{th}} \\ 0 & \text{otherwise} \end{cases}, \quad V_{\text{mem}} \leftarrow V_{\text{reset}} \text{ after spike.} \quad (3)$$

The synaptic input is filtered through an exponential kernel:

$$I_{\text{syn}}(t) = \sum_j w_j \sum_{t_j^f} \exp\left(-\frac{t - t_j^f}{\tau_s}\right) \quad (4)$$

where t_j^f are the firing times of presynaptic neuron j and w_j is its synaptic weight.

2.3. Spike-Timing-Dependent Plasticity (STDP)

STDP is the biologically observed learning rule in which synaptic strength is modulated by the relative timing of pre- and post-synaptic spikes [2]. The weight update for synapse j is:

$$\Delta w_j = \begin{cases} A_+ \exp\left(-\frac{\Delta t}{\tau_+}\right) & \text{if } \Delta t > 0 \quad (\text{pre before post: LTP}) \\ -A_- \exp\left(\frac{\Delta t}{\tau_-}\right) & \text{if } \Delta t < 0 \quad (\text{post before pre: LTD}) \end{cases} \quad (5)$$

where $\Delta t = t_{\text{post}} - t_{\text{pre}}$, and A_+, A_-, τ_+, τ_- are positive constants controlling potentiation and depression amplitudes and time windows. Weights are bounded: $w_j \in [w_{\min}, w_{\max}]$, linking directly to the memristor conductance model in Eq. (1).

Link to memristor: Eq. (5) is a discretized, spike-triggered version of Eq. (1). A positive voltage pulse (pre fires before post) increases conductance (LTP); a negative pulse does the reverse (LTD). This is why the FPGA implementation of the weight-update crossbar is structurally an STDP accelerator whose topology mirrors a memristive crossbar—it is described as *memristor-inspired* rather than a memristor emulator throughout this proposal.

2.4. The Credit-Assignment Problem: Reward-Modulated STDP as the Primary Rule

A central design decision of this thesis follows directly from a limitation of Eq. (5). Pure STDP is *Hebbian*: it strengthens or weakens synapses based only on the relative timing of pre- and post-synaptic spikes, with no reference to task outcome. There is no term in the rule that encodes “this spike pattern preceded a collision, suppress it.” Consequently, once an SNN is released from surrogate-gradient pretraining into pure online STDP, weight drift is driven by the correlation structure of the input spike trains—not by navigation success. Surrogate-gradient pretraining establishes a good starting point, but pure STDP has no mechanism to keep the network near that point in a *task-relevant* direction. This is precisely why Lobov et al. [8] and, more explicitly, Juarez-Lora et al. [9] use *reward-modulated* STDP (R-STDP), in which a global reward/error signal gates an eligibility trace before it is consolidated into Δw_{ij} .

Because the central hypothesis (H1) is a claim about *task-relevant* recovery, this thesis adopts **R-STDP as the primary learning rule**, and retains pure Hebbian STDP as an ablation used to demonstrate exactly why reward modulation is necessary:

1. **(Primary plan) R-STDP.** An eligibility-trace buffer $e_{ij}(t)$ per synapse is updated by the Hebbian term of Eq. (5) on every spike pair, and a scalar reward/collision signal $r(t)$ (e.g. $r = -1$ on collision, small positive shaping reward for progress toward goal) is broadcast to all synapses:

$$\frac{de_{ij}}{dt} = -\frac{e_{ij}}{\tau_e} + \Delta w_{ij}^{\text{STDP}}, \quad \Delta w_{ij} = \eta r(t) e_{ij}(t). \quad (6)$$

The FPGA design contains the per-synapse Δt array and LUT; R-STDP adds one eligibility-trace register and one scalar multiply per synapse, which is a modest hardware extension.

2. **(Ablation) Pure STDP.** The same network with the reward gate removed ($\Delta w_{ij} = \Delta w_{ij}^{\text{STDP}}$ applied directly). This ablation is expected *not* to recover the navigation policy in a task-relevant way; any residual effect is a *representation-level* change (e.g. sensory-layer receptive-field drift in response to the post-shift input statistics) rather than policy adaptation. Contrasting R-STDP against this ablation is what attributes recovery specifically to reward-gated plasticity.

Risk: STDP Credit Assignment

If reward modulation were omitted, H1 would risk failing for reasons unrelated to memristor-style plasticity (i.e. because Hebbian drift is task-agnostic), which would muddy the thesis conclusion regardless of how well the FPGA or simulation work is executed. Adopting R-STDP as the primary rule removes this confound. The eligibility-trace and reward-broadcast machinery is small in both software and hardware (one register + one multiply per synapse), so it is committed to from the start rather than deferred; the pure-STDP ablation is cheap because it is the same code path with the reward gate disabled.

3. System Architecture

The full system has three layers: simulation environment, neural controller, and FPGA co-processor.

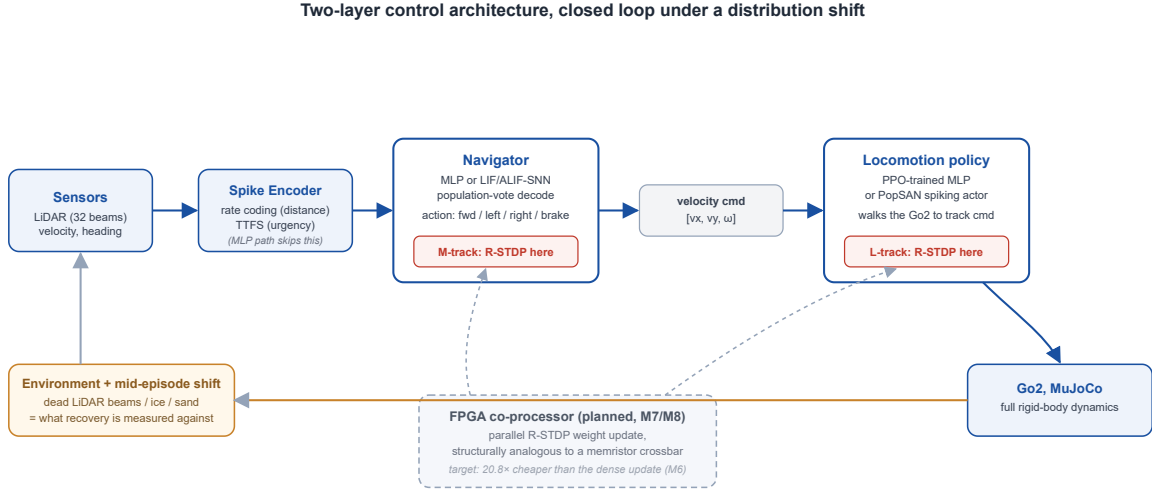


Figure 1: System Architecture showing the closed-loop control and FPGA co-processor.

3.1. Baseline and Ablation Controllers (Comparison Group)

Four controllers are used to establish a fair and rigorous comparison:

1. **MLP-DNN (frozen):** A multi-layer perceptron trained offline via supervised imitation learning on expert trajectories. After training, weights are fixed. This isolates the effect of online plasticity.
2. **MLP-DNN (Online-Updating):** An MLP that performs continuous online learning via stochastic gradient descent (SGD) on a sliding window of recent observations. *Loss target:* to avoid the two obvious failure modes — (i) pure self-supervision on the network’s own past actions, which collapses to a fixed point with no corrective signal, and (ii) continued access to expert labels during Phase 2, which would give this baseline an unfair advantage the SNN does not have — the online loss is a one-step TD-bootstrapped value/advantage target computed from the robot’s own collision and progress-to-goal signals (the same signals used for the R-STDP reward broadcast in Section 2.4). This keeps the comparison fair: both adapting controllers learn from the same environment-derived feedback, differing only in *how* that feedback is consolidated into weight changes.
3. **SNN (frozen, ablation):** The same LIF-SNN architecture and surrogate-gradient-pretrained weights as the proposed controller, but with plasticity *disabled* during evaluation. This isolates the contribution of spiking/LIF temporal dynamics (which alone may provide some robustness via low-pass filtering, per H4) from the contribution of online plasticity (H1). Without this ablation, any advantage of the SNN–R-STDP controller cannot be attributed to plasticity specifically.
4. **RL Policy (PPO):** A proximal policy optimization agent with an MLP policy network, trained in PyBullet. This is the strongest fixed-policy baseline and represents the current state of practice for learned robotic control. Its policy is frozen post-training for this comparison, so it is not included in the recovery-time comparison but serves as an absolute performance ceiling for Phase 1.

Why not CNN? CNNs are designed for grid-structured spatial data (images). The navigation task uses LiDAR distance readings and pose vectors—a flat, non-spatial observation space better suited to an MLP. Using CNN as the baseline would be testing the wrong inductive bias, not the right one.

3.2. Proposed Controller: LIF-SNN with R-STDP

1. **Input encoding:** LiDAR readings and velocity are encoded as spike trains using rate coding (firing rate proportional to stimulus intensity) for distance channels and time-to-first-spike (TTFS) coding for urgency signals.
2. **Network:** A fully connected LIF-SNN (3–4 layers, ~ 512 – 1024 neurons) implemented in SNN-Torch [7]. Initial weights are pretrained via surrogate-gradient backpropagation to match the MLP-DNN, then *released* to online R-STDP (Section 2.4) adaptation during evaluation.
3. **Output decoding:** Population vote over output neurons, one population per discrete action (left, right, forward, brake).
4. **Online adaptation:** At each timestep, spike times and the scalar reward signal are sent to the FPGA, which computes Δw_{ij} in parallel across all synapses and returns updated weights before the next action.

3.3. FPGA Synaptic Update Coprocessor

The key architectural claim of this thesis is that the STDP weight-update step has the same topology and update locality as a memristor crossbar, and that this parallelism can be hardware-exploited. On an ideal crossbar, $N \times M$ synapses update *simultaneously* in $O(1)$ time. On a CPU, this is an $O(N \cdot M)$ sequential (or vectorized-but-still-bandwidth-bound) loop. An FPGA sits between these extremes: it can update many synapses per cycle, but—unlike an analog crossbar—it is limited by the number of physical DSP/multiplier resources available, so true $O(1)$ parallelism only holds up to that resource count.

The FPGA implementation (targeting a Xilinx Zynq-7000 or Intel Cyclone V) consists of:

- **Spike-time registers:** One register per neuron, holding the most recent spike timestamp t_j^f in fixed-point format (16-bit, 0.1 ms resolution).
- **Δt computation array:** Parallel subtractors computing $\Delta t_{ij} = t_i^{\text{post}} - t_j^{\text{pre}}$ for a tile of synapses simultaneously (see resource discussion below).
- **Exponential lookup table (LUT):** Pre-computed STDP kernels stored in BRAM. The STDP window ($\tau_+, \tau_- \approx 20$ – 40 ms) combined with 0.1 ms timestamp resolution spans 200–400 raw ticks; these are mapped onto 64 LUT bins via *logarithmic* (not linear) quantization of $|\Delta t|$, since the exponential kernel in Eq. (5) varies most steeply for small $|\Delta t|$ and is nearly flat for large $|\Delta t|$. This bin layout is validated against the floating-point kernel in simulation before synthesis (target: $< 2\%$ relative error in Δw).
- **Weight update array:** Parallel multiply-accumulators applying the LTP/LTD rule and the eligibility-trace update of Eq. (6), and clamping to $[w_{\min}, w_{\max}]$.
- **AXI4-Stream (DMA) interface:** Implements Direct Memory Access for burst-transferring entire vectors of spike states between CPU RAM and the FPGA, avoiding the per-register transfer overhead of AXI4-Lite.

Risk: Resource-Constrained “Parallelism”

A fully connected 512×512 synapse layer requires on the order of 2.6×10^5 parallel multiply-accumulate units for true one-cycle updates. A Zynq-7020 provides only 220 DSP slices. The realistic

design therefore **time-multiplexes** a bank of processing elements (PEs) across synapse tiles rather than instantiating one PE per synapse—this is not a true $O(1)$ crossbar, and the proposal should not claim it is. Additionally, the AXI-DMA round trip per timestep has a fixed latency floor that, for small networks, can dominate the actual update compute. Consequently:

- The 10–50 $\times$ speedup claim (H3) applies to the *inner weight-update kernel only*, not the end-to-end control loop including DMA transfer—this hedge is made explicit as a named metric in Section 4.3.
- Phase 4 includes a **network-size sweep** (e.g. 64, 128, 256, 512 synapses-per-layer) reporting kernel time and end-to-end (including DMA) time for both CPU and FPGA, so the crossover point at which FPGA acceleration becomes worthwhile is an empirical result rather than an assumption. At small sizes, CPU/GPU vectorized BLAS may simply win, and that is a valid finding.

This architecture is *structurally analogous* to a memristive crossbar: each PE corresponds to (a time-multiplexed share of) one synapse, spike arrival is the analog of a voltage pulse, and the weight register is the analog of the conductance state g . Unlike Eq. (1), the FPGA operates in discrete time on quantized spike events and is resource-bounded rather than truly parallel; the term “memristor-inspired” is used throughout to reflect this.

4. Experimental Setup

4.1. Simulation Environment: PyBullet

All experiments run in PyBullet [12], a real-time physics simulation library. The scenario:

- **Robot:** Ground robot with 360° LiDAR (32 beams, 8 m range) and an IMU. The observation space is deliberately kept flat and non-spatial (LiDAR ranges + pose/velocity) to justify the MLP inductive bias (Section 3.1); no camera is used.
- **Environment:** Enclosed arena (10 $\times$ 10 m) with randomized static obstacles (boxes, cylinders) and dynamic obstacles moving at 0.2–0.5 m/s, i.e. below the robot’s forward speed so that evasion is always kinematically possible (a faster obstacle could catch the robot from behind, making some collisions unavoidable for *any* controller and compressing the inter-controller differences the study measures).
- **Task:** Reach a goal position without collision. Episode length: 60 s.
- **Environment shift:** At $t = 30$ s, the obstacle density doubles and three new dynamic obstacles are injected. This is the critical test of adaptivity.

Two-layer control and robot platform. The neural controller is the *high-level navigator*: it maps sensor observations to a body-velocity command $[v_x, v_y, \omega]$ (realized as the discrete actions forward/left/right/brake). This interface is platform-agnostic and decouples the navigation contribution from low-level actuation. The core comparative study runs on a fast kinematic-unicycle model for statistical throughput. For realistic validation and a hardware demonstration, the same velocity commands drive a **Unitree Go2** quadruped: in simulation via a convex-MPC locomotion controller, and on the physical robot via the onboard SDK’s velocity-command (“sport”) mode—so no bespoke gait is deployed to hardware. The locomotion layer is developed in parallel and does not affect the plasticity results.

4.2. The Key Experiment: Online Adaptation Under Distribution Shift

This is the experiment that differentiates this thesis from prior work. The protocol:

1. Run all controllers (MLP-DNN frozen, MLP-DNN online, SNN frozen ablation, SNN-R-STDP, and PPO) for the first 30 s with a fixed environment (Phase 1).

2. At $t = 30$ s, inject the environment shift without stopping the episode (Phase 2).
3. Measure **performance recovery time**: how many seconds until each controller’s collision-free navigation rate returns to within 10% of its Phase 1 baseline.
4. The MLP-DNN (frozen), the SNN (frozen ablation), and the PPO policy *cannot* adapt online. The evaluation focuses on comparing recovery times between the SNN-R-STDP controller and the MLP-DNN (online) controller, with the frozen variants providing the non-adapting reference level that defines “no recovery.”

4.3. Evaluation Metrics

Metric	Definition	Unit
Response latency	$T_{\text{resp}} = T_{\text{action}} - T_{\text{sensor}}$	ms
Energy efficiency	Synaptic operations: $\text{SynOps} = \sum_t \sum_{ij} s_i(t) \cdot \mathbf{1}_{w_{ij} \neq 0}$	ops/step
Adaptation recovery	τ_{rec} : seconds to recover 90% task performance after shift	s
Robustness	Success rate under additive Gaussian noise $\sigma \in \{0, 0.1, 0.3, 0.5\}$	%
FPGA kernel speedup	$S_{\text{kernel}} = T_{\text{CPU}}/T_{\text{FPGA}}$ for the weight-update kernel only	$\times$
FPGA end-to-end speedup	$S_{\text{e2e}} = T_{\text{CPU}}/(T_{\text{FPGA}} + T_{\text{DMA}})$, reported per net-work size	$\times$

Why SynOps and not $\sum |\Delta w|$? SynOps counts how many multiply-accumulate operations the network actually performs per timestep, accounting for spike sparsity. It is the standard neuromorphic efficiency metric [5] and maps directly to hardware energy because neurons that do not spike consume no compute. $\sum |\Delta w|$ has no physical interpretation and cannot be compared across architectures.

4.4. Software Stack

- **SNN framework:** SNN_Torch [7] (Python, PyTorch backend)
- **RL baseline:** Stable-Baselines3 (PPO) [11]
- **Simulation:** PyBullet [12]
- **FPGA:** Xilinx Vivado / Intel Quartus (target: Zynq-7020 or Cyclone V)
- **Hardware interface:** PyNQ (Python-to-FPGA AXI DMA bridge)
- **Logging:** Weights & Biases for experiment tracking

5. Memristor-Inspired Update Model: Fidelity and Scope

To ground the software and FPGA models in physical device behavior, the Biolek memristor model [6] is used as a *conceptual reference*, not as a target for direct emulation:

$$\frac{dx}{dt} = \mu_v \frac{R_{\text{on}}}{D^2} i(t) f(x) \quad (7)$$

where $x \in [0, 1]$ is the normalized state variable (analogous to normalized conductance g), μ_v is the ion mobility, D is the device length, and $f(x)$ is a window function that prevents the state from leaving $[0, 1]$. The STDP rule in Eq. (5) is a spike-triggered, discrete-time *analogy* to Eq. (7): a positive Δt corresponds qualitatively to a potentiating pulse, and a negative Δt to a depressing pulse. The thesis does not claim quantitative correspondence between Δw_{ij} and Δx in Eq. (7); the correspondence is structural (bounded state, history dependence, locality of update).

The three claimed memristive *properties* are reproduced structurally as follows (software model and FPGA emulation columns describe analogous, not identical, mechanisms):

Memristive Property	Software Model	FPGA Implementation
State retention	Weight registers persist between steps	BRAM weight array
Nonlinearity & saturation	$w \in [w_{\min}, w_{\max}]$ clamp	Saturating adder
Stochasticity	Additive noise $\epsilon \sim \mathcal{N}(0, \sigma_w^2)$	LFSR noise unit

6. Expected Results and Hypotheses

Hypotheses

- H1. Adaptation:** The SNN-R-STDP controller will recover 90% task performance after the environment shift within $\tau_{\text{rec}} < 5$ s, outperforming both the frozen MLP-DNN and the online-updating MLP-DNN. The pure-STDP ablation is expected *not* to achieve task-relevant recovery; any effect it shows is confined to a measurable reorganization of sensory-layer representations (quantified via receptive-field drift), while the frozen-SNN ablation shows no such reorganization. This contrast attributes recovery specifically to reward-gated plasticity rather than to Hebbian drift or LIF dynamics alone.
- H2. Energy:** SNN SynOps will be $3\text{--}10\times$ lower than DNN multiply-accumulate operations at equal pre-shift performance, due to spike sparsity.
- H3. Latency:** FPGA parallel weight update via AXI-DMA will be $10\text{--}50\times$ faster than CPU computation for the weight-update *kernel* alone, with the crossover point (network size at which FPGA beats CPU/GPU end-to-end) determined empirically by the Phase 4 sweep.
- H4. Robustness:** SNN success rate will degrade more gracefully under increasing sensor noise σ , due to the temporal integration inherent in LIF dynamics acting as a low-pass filter. The frozen-SNN ablation isolates whether this robustness comes from LIF dynamics alone or requires plasticity.

The hypotheses are intentionally falsifiable. If H1 does not hold even for R-STDP, the thesis conclusion is that conventional DNNs with fast online retraining are a better practical strategy than online reward-modulated plasticity for this task class—which is itself a valuable finding.

7. Project Timeline and Budget

7.1. Phased Timeline (6–9 months)

Phase	Duration	Deliverables
1	Weeks 1–4	Literature review finalized; PyBullet environment built; MLP-DNN baselines (frozen and online, with TD-based online loss) trained and evaluated
2	Weeks 5–11	SNN implemented in SNN_Torch; surrogate-gradient pretraining; frozen-SNN ablation evaluated; pilot experiment : R-STDP vs. pure-STDP vs. frozen-SNN recovery under the shift (the go/no-go gate before hardware)
3	Weeks 12–17	PPO baseline trained; full comparative experiment across all metrics; environment-shift protocol run for all five controllers; robustness and energy sweeps (H2, H4)
4	Weeks 18–24	FPGA R-STDP accelerator designed and synthesized (resource-aware, time-multiplexed PE design); LUT bin layout validated against the software golden reference; hardware-in-loop timing measurements; network-size sweep for kernel and end-to-end FPGA vs. CPU/GPU speedup
5	Weeks 25–30	Thesis writing; figure generation; revision; presentation prep

7.2. Budget Estimate

Item	Cost (EUR)	Notes
Xilinx Zynq-7020 SoC board (e.g. Digilent Arty Z7-20)	180–220	AXI + ARM core; PyNQ compatible
<i>Alternative:</i> Intel Cyclone V (DE10-Nano)	120–150	Budget option; HPS for Python side
LiDAR sensor (RPLIDAR A1M8, if physical demo)	100–130	Optional; simulation covers this
Differential-drive robot chassis (if physical demo)	80–120	e.g. Waveshare RaspiCar or similar
GPU cloud compute (Vast.ai, 50 h at ~€1.2/h)	60–80	For PPO training and SNN pretraining
Miscellaneous (cables, SD cards, USB hubs)	20–30	
Total (simulation-only + FPGA)	260–330	
Total (FPGA + physical robot)	440–580	

Both scenarios are well within a 1000–1500 EUR budget, leaving substantial headroom.

8. Related Work and Further Reading

The following papers and resources define the landscape this thesis operates within. They are grouped by sub-topic for targeted reading.

Memristive Devices and Neuromorphic Hardware

- Strukov et al. (2008), *Nature*—“The missing memristor found.” Foundational paper establishing memristors as circuit elements [1].
- Biolek et al. (2009)—SPICE memristor model with window functions [6].
- Merolla et al. (2014), *Science*—IBM TrueNorth: 4096 cores, 1 million neurons, 256 million synapses at 70 mW [4].
- Davies et al. (2018), *IEEE Micro*—Intel Loihi: on-chip STDP, first programmable neuromorphic chip with online learning [5].
- Shooshtari et al. (2025), *Advanced Intelligent Systems*—Review of memristors as the hardware bridge between in-memory computing and SNNs.

Spiking Neural Networks: Theory and Training

- Maass (1997), *Neural Networks*—Networks of spiking neurons as the third generation of neural network models [3].
- Bi & Poo (1998), *J. Neuroscience*—Original STDP experimental measurement in hippocampal neurons [2].
- Lee et al. (2016), *Front. Neuroscience*—Training deep SNNs via surrogate gradients; the method used for SNN pretraining here.
- Eshraghian et al. (2023), *Proc. IEEE*—SNN-Torch framework paper; the primary software tool for this thesis [7].

Neuromorphic Robotics: Most Directly Relevant

- Lobov et al. (2020), *Front. Neuroscience*—STDP-controlled mobile robot with memristive devices; nearest predecessor to this work [8].
- Juarez-Lora et al. (2022), *Front. Neurorobotics*—R-STDP for robotic arm control under changing joint friction; demonstrates online adaptation and motivates the reward-modulated rule adopted in Section 2.4 [9].
- Abdelrahman et al. (2024/2025), *Int. J. Robotics Research*—Convolutional SNN for obstacle avoidance on a Kinova Gen3 arm; tested in Gazebo and real world.
- Stroobants et al. (2024), *arXiv:2411.13945*—First full neuromorphic attitude control SNN deployed on a real Crazyflie quadrotor.
- Supic & Stewart (2022), *ACM*—SNN adaptive control for robot arms; 15% faster task completion with online learning over fixed policies.

FPGA Neuromorphic Acceleration

- Qiao et al. (2015), *Front. Neuroscience*—ROLLS neuromorphic chip with embedded FPGA interface for spike routing; architectural reference [10].
- Neil & Liu (2014), *IEEE BioCAS*—Minitaur: FPGA-based SNN simulator with on-chip STDP; close analog to the FPGA module proposed here, including its resource-multiplexing approach.
- Prajapati et al. (2025), *arXiv:2507.20998*—Memristive SNN with supervised in-situ STDP; most recent circuit-level implementation reference.

Reinforcement Learning Baseline

- Schulman et al. (2017), *arXiv:1707.06347*—PPO algorithm used for the RL baseline policy.
- Raffin et al. (2021), *JMLR*—Stable-Baselines3 library [11].

9. Conclusion

This thesis proposes a concrete, falsifiable comparative study of neuromorphic vs. conventional robotic control, grounded in three interlocking contributions: (1) a software SNN with biologically grounded, reward-modulated STDP, evaluated with frozen-SNN and pure-STDP ablations that separate the contributions of spiking/LIF dynamics, Hebbian drift, and reward-gated on-line plasticity under environment shift, (2) an FPGA coprocessor whose update topology is structurally analogous to a memristive crossbar—explicitly scoped as resource-constrained and time-multiplexed rather than truly $O(1)$ —and (3) a quantitative evaluation framework using standard metrics (SynOps, recovery time, kernel and end-to-end FPGA speedup) that allows direct comparison with the existing literature.

The project is achievable within a 6–9 month timeline and a budget of 440–580 EUR including physical hardware, or under 350 EUR for a simulation-only study. The FPGA component is not decorative—it is the hardware argument for why memristor-inspired, in-memory synaptic computation is worth building, even within the resource limits of a commodity FPGA: it makes the parallel-update structure of synaptic learning physically instantiable without fabricating nanoscale devices.

References

- [1] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, “The missing memristor found,” *Nature*, vol. 453, pp. 80–83, 2008.
- [2] G. Q. Bi and M. M. Poo, “Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type,” *Journal of Neuroscience*, vol. 18, no. 24, pp. 10464–10472, 1998.
- [3] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [4] P. A. Merolla *et al.*, “A million spiking-neuron integrated circuit with a scalable communication network and interface,” *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [5] M. Davies *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [6] Z. Biolek, D. Biolek, and V. Biolkova, “SPICE model of memristor with nonlinear dopant drift,” *Radioengineering*, vol. 18, no. 2, pp. 210–214, 2009.
- [7] J. K. Eshraghian *et al.*, “SNN-Torch: Accelerating neuromorphic engineering with a deep learning algorithm library,” *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1–19, 2023.
- [8] S. A. Lobov *et al.*, “Spatial properties of STDP in a self-learning spiking neural network enable controlling a mobile robot,” *Frontiers in Neuroscience*, vol. 14, p. 88, 2020.
- [9] A. Juarez-Lora *et al.*, “R-STDP spiking neural network architecture for motion control on a changing friction joint robotic arm,” *Frontiers in Neurorobotics*, vol. 16, 2022.
- [10] N. Qiao *et al.*, “A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128K synapses,” *Frontiers in Neuroscience*, vol. 9, p. 141, 2015.
- [11] A. Raffin *et al.*, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

- [12] E. Coumans and Y. Bai, “PyBullet, a Python module for physics simulation for games, robotics and machine learning,” <http://pybullet.org>, 2016–2021.